

Week 14 • Own it at the exam

Technical Interaction Design • ITU • Autumn 2026

Built 11 September 2026 • commit bc3510e

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-14.pdf>

CONTENTS

Report Structure

- Sections

Exam Structure

- On the day
- Giving the demo
- What you get asked

Report Structure

Source: [Report-Structure.md](#) — something unclear? [Open an issue](#), or send a pull request.

The report is written **for a fellow technical interaction designer**. It is not a scientific report — no citations, no theory. Just your solutions, and why they are the ones you chose.

The body carries **decisions and their consequences**. Artefacts — wireframes, raw session notes, anything you produced along the way — go in the appendix. An examiner reading this on a plane wants to know what you learned and what you changed because of it; they will look at the screens if they want to check you.

Which means it has to stand on its own. Link to your Figma if you have one, but a link is a supplement, never the only copy: if it is needed to follow the argument, it is in here.

Sections

- **Title page** — project name, team members, names and emails.
- **Project description** (one page) — what you set out to achieve, for whom, and the one twist that makes it yours rather than a generic to-do.
- **The user study** — your population, how many people you spoke to, the themes that came out of it, and the prioritised tasks you derived. The deliverable you handed in, as it ended up rather than as it started.
- **What the testing changed** (one or two pages) — you ran think-alouds on the **wireframes**, before any of it was built. So: which tasks you asked people to do, what went wrong, and what you redrew because of it. Findings and decisions here; the screens themselves and your session notes go in the appendix.

A finding nobody acted on is worth reporting too, as long as you say why not.

- **The data model** (one page) — the ER diagram, and a paragraph on the parts of it that were not obvious: what you made a pointer and what you made a relation, where sharing lives, anything you modelled one way and then changed.
- **Visual design and accessibility** (one or two pages) — the design language you chose and why, plus the accessibility audit and what it made you fix.
- **Implementation**
 - Link to the GitHub repository
 - Link to your Back4App database — invite the examiner accounts listed on learnIT
 - How the examiners run your code. Normally your repo's `README.md` already explains this. But if anything secret is needed — the contents of an `.env` file, app keys, configuration that is deliberately not committed — put it in the report.
 - If you have created accounts for the examiners to log in with, say so and give the credentials. (This makes sense for some applications and not others.)
- **Reflection** (two pages max) — looking back at the project:
 - What worked well?
 - What did using your own app teach you that the testing did not?
 - What could have been done better — by the team, and by the TID staff?
- **Appendix** — the wireframes, the think-aloud notes, and anything else you want on record. No page limit, and nobody expects it to be polished.

Something unclear in this section? [Open an issue](#).

Exam Structure

Source: [Exam-Structure.md](#) — something unclear? [Open an issue](#), or send a pull request.

On the day

- **Demo** — 5–7 minutes, maximum.
- **Mini presentation** — 5–7 minutes: corrections or completions to the report, late changes to the code, and lessons learned. Late changes matter: realistically we will not have looked at the code more than a week before the oral exams, so anything you fixed or finished after submission is worth naming.
- **Group discussion** — 10 to 30 minutes.
- **Individual examination** — 15 minutes.
- **Feedback.**

The individual examination is where the course's deal is settled: you may have used AI as much as you liked, and you own every line when we meet.

Something unclear in this section? [Open an issue](#).

Giving the demo

Five to seven minutes is short, and the failure modes are consistent — these came out of watching previous cohorts present:

- **Open with a slide that says what the idea is.** Thirty seconds of “here is who this is for and what it does” before anything moves on screen.
- **Have real data in the system.** An app demoed with `asdf` and `test test` looks unfinished whatever the code is like.
- **Have a story, and rehearse it.** Not a feature tour — one path through the app that a real person would actually take. Steve Jobs' keynotes and Google's product demos are worth watching for exactly this.
- **Reset your state before you start.** The profile you have been developing against is full of debris. Clear it, or make a clean account.

Something unclear in this section? [Open an issue](#).

What you get asked

About your code

- Where is *this particular feature* implemented?
- What does *this code* do?
- How could *this code* be written better?
- This bug we just saw in the app — where would you start fixing it?
- *This piece of code* — when is it called?
- This feature — how is it modelled in the database?
- Which part of the code did you write yourself, and are most proud of?

About your UI design

- What are the limitations of the design on *this particular* screen?
- Which visual design principles are used on *this particular* screen?

About design process

- How do you organise a usability test?
- How do you prioritise what a usability test finds?

About application architecture

- What kinds of computation need to run *on the server*? How do you do that in Parse?
- What is the N+1 select problem, and how do you avoid it in Parse?

React

- How is routing implemented in React?
- How do you protect a route so it is not available to a user who is not logged in?
- What are hooks, and why? Which are the most used, and how do they work?
- How do you run code when a component is created?
- How can you reuse code with a custom hook? Do you have an example from your codebase?

Parse

- How do you model a one-to-many relationship in Parse? An example from your project?
- How do you secure your application's data in Parse?

Every note in [Lectures/Technical/](#) also ends with its own **Exam Questions** section. Those test the material; the ones above test *your project*.

Something unclear in this section? Open an issue.